

DELFT UNIVERSITY OF TECHNOLOGY

INTEGRATION PROJECT SYSTEMS AND CONTROL
SC42035

Final Report

1-DoF Helicopter Pitch Control

Group 10 (Helicopter 2A):

Melis Orhun (4912071)

Sven Rutgers (4600150)

June 20, 2024



Table of Contents

Introduction	2
1 Setup description	2
1.1 Sensor calibration	3
2 Modelling	3
2.1 Schematic overview	3
2.2 Nonlinear equations of motion	5
2.3 Linear equations of motion	6
3 System identification	7
3.1 Identification strategy	7
3.2 Excitation signals	8
3.3 Grey-box Model Estimation	8
3.4 Model Validation	9
3.5 Results	9
4 Model estimation	10
5 Control design	13
5.1 Objectives	13
5.2 Choice of architecture	13
5.3 Linear-Quadratic-Integral (LQI) control	14
5.4 Model Predictive Control (MPC)	17
Discussion	19

Introduction

This report contains the methodology and results of the project for the TU Delft course Integration Project for Systems and Control (SC42035). The goal is to make the helicopter hover stationary at a specific angle and to make the pitch angle follow a specified reference. First, the helicopter setup and the calibration of its sensors are described. The dynamical system is modeled using equations of motion so that the system can be identified using a grey-box approach. The fidelity of the identified model is then evaluated. Different control strategies are discussed, leading to the design of two controller types. The first utilizes a Linear-Quadratic Regulator (LQR)/Linear-Quadratic-Integral control (LQI), and the second employs a Model Predictive Control (MPC) approach. These controllers are implemented, and their performance is evaluated. Finally, an overall discussion of the entire project is provided.

1 Setup description

For this project, a simplified one degree of freedom (1-DoF) helicopter system is utilised. This system is shown below in Figure 1.



Figure 1: 1-DoF helicopter setup

This setup consists of a rotating horizontal beam, fixed in height at its centre point to the top of a vertical pole. At the centre of rotation, a vertical counterweight is attached, lowering the helicopter's centre of gravity. Two DC motors with propellers are attached to both ends of this beam. Voltage can be applied to these motors to control the movement of the helicopter around its axes of freedom. For pitch control, two of the four sensors are used: one that measures the pitch angle α and one that measures the angular velocity of the elevation/pitch motor ω .

This setup can freely rotate its pitch, which is the movement around the lateral axis, sometimes called elevation. Normally, the system is also able to rotate around the vertical axis (azimuth/yaw movement). However, this movement is prevented by a metal pin, which can be seen at the right of the top of the vertical pole in Figure 1. After fixing the yaw position, only one degree of freedom, the pitch, remains for the helicopter body. The rotational movements, pitch, roll, and yaw, are visualised on the next page in Figure 2 for clarity.

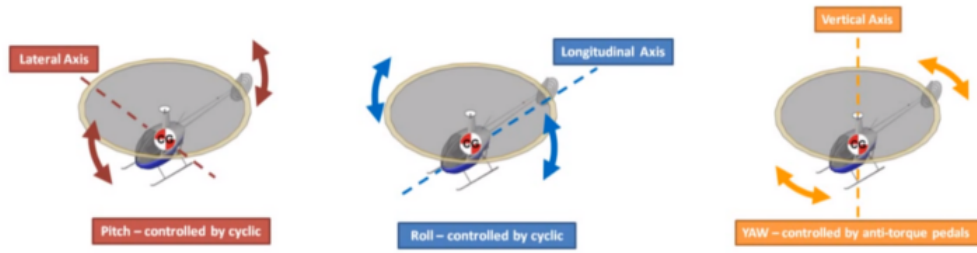


Figure 2: Three axes of rotation [1]

The entire system can be described by the pitch angle α and motor angular velocity ω . An additional state, pitch angular velocity $\dot{\alpha}$ is required to link control input u , via ω , to pitch angle α . The objective is to identify the dynamical model of the system to control the pitch motor (left in Figure 1) to hover stationary at an angle and to make the attitude of the beam follow a specified reference, while improving damping.

1.1 Sensor calibration

The sensors must be calibrated to ensure the measured output values match reality. This is necessary for proper identification, which, in effect, enables better control performance and should not be neglected. To perform the calibration, the protractor at the top of the vertical setup can be used to read the actual angle values.

Since the objective is to control the pitch angle, the corresponding sensor provides the most important measurements. The pitch angle sensor provides values in radians, which can easily be converted into degrees by multiplying by $180^\circ/\pi$. There was a small offset between the actual angle and the measured output. This is adjusted by subtracting this offset from the pitch sensor output values. No scaling is necessary.

The calibration of the other sensor, which measures the angular velocity of the motorised propeller, is more challenging. There is no straightforward method to measure the actual angular velocity for comparison with the measured values. Since the maximum measured value was approximately 400, it appears that the output is in terms of revolutions per minute (RPM). As this angular velocity is just an intermediate step in controlling the pitch angle, perfect calibration is not essential. If this were necessary, the actual angular velocity could, for example, be achieved by filming the actual movement in slow motion using a camera with a known frame rate. However, it is important to understand the relationship between the normalised input voltage and the affected states. This relationship will be described in Chapter 2 and finalised in Chapter 3.4.

2 Modelling

As mentioned previously, achieving good control performance necessitates a high-fidelity system model. Before fully identifying the system, it is beneficial to gather as much information about its dynamics as possible. This a priori knowledge can then be exploited in grey-box model estimation to efficiently achieve a better more reliable model than would be possible with solely data oriented approach like black-box estimation (without prior knowledge). In this chapter, the equations of motion are derived for the three states that describe the system behaviour, pitch angular velocity $\dot{\alpha}$, pitch angle α and motor angular velocity ω .

2.1 Schematic overview

Firstly, the symbol definitions and conventions, for example indicating positive directions, are described. The equations of motion will be derived using the sum of moments around centres of rotation. It might be necessary to obtain initial guesses for the unknown system parameters. Therefore, the sums of moments are described by as many real-life parameters as possible. The used symbols are described in Table 1 on the next page. The symbols used to describe the state space are shown at the top.

Symbol	Definition [unit]
$\ddot{\alpha}$	Pitch angular acceleration [rad/s ²]
$\dot{\alpha}$	Pitch angular velocity [rad/s]
α	Pitch angle [rad]
$\dot{\omega}$	Motor angular acceleration [rad/s ²]
ω	Motor angular velocity [rad/s]
u	Motor input voltage [V]
$a_i; b_1$	System coefficients to be identified
CoR	Centre of rotation [-]
CoM	Centre of mass [-]
F_g	Gravitational force [N]
F_f	Friction force [N]
F_L	Lift force [N]
g	Gravitational constant [m/s ²]
J_α	Moment of inertia pitch [kg·m ²]
J_ω	Moment of inertia propeller [kg·m ²]
k_i	(Unknown) coefficient
l_m	Length mass arm [m]
l_L	Length lift arm [m]
m	Mass [kg]
M_D	Moment of propeller drag [N·m]
M_f	Moment of F_f [N·m]
M_g	Moment of F_g [N·m]
M_L	Moment of F_L [N·m]
M_u	Moment of DC motor torque [N·m]

Table 1: Symbol definitions

Now, a schematic representation of the setup can be created to show the positive directions. The model for the body of the 1-DoF helicopter is shown below in Figure 3. To more clearly show the positive direction of the propeller angular velocity ω , a top view is provided in Figure 4. These conventions will be upheld for future computations using these variables. The yet unmentioned variables from Table 1 will be introduced in Section 2.2 or derived using the variables from Figures 3 and 4.

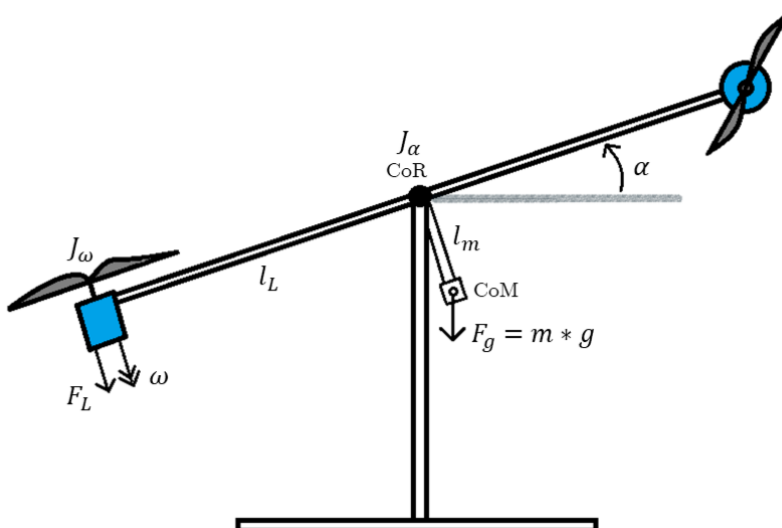


Figure 3: Schematic representation of 1-DoF helicopter

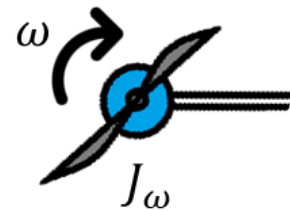


Figure 4: Top view of schematic pitch propeller

2.2 Nonlinear equations of motion

The dynamics of the system are described to provide the grey-box system estimation with a priori knowledge. The symbols described in Table 1 of Section 2.1 will be used to compute the sum of moments of the pitch propeller and around the centre of rotation of the body of the 1-DoF helicopter.

To find the differential equation related to the pitch angular velocity $\dot{\alpha}$, the sum of moments around the centre of rotation (CoR) is derived, starting with the moment created by the gravitational force. Here, the moment has a negative direction and the effective arm d is dependent on the pitch angle α and the length of the mass arm l_m , as is shown in Equation 1.

$$\begin{aligned} M_g &= -F_g \cdot d \\ F_g &= m \cdot g \\ d &= l_m \cdot \sin \alpha \\ M_g &= -m \cdot g \cdot l_m \cdot \sin(\alpha) \end{aligned} \tag{1}$$

In general, the formula of Equation 2 can be used to model the drag and lift of a wing. Here, c_l is a constant empirically derived coefficient, ρ is the air density, A is the contact surface, and v is the wing velocity. This is adjusted to angular velocity by applying $v = \omega \cdot r$, where r is the radius. The (unknown) constants can be combined into a single unknown coefficient k .

$$\begin{aligned} F_L &= \frac{1}{2} \cdot c_l \cdot \rho \cdot A \cdot v^2 \\ F_L &= \frac{1}{2} \cdot c_l \cdot \rho \cdot A \cdot r^2 \cdot \omega^2 \\ F_{L/D} &= k_i \cdot \omega^2 \end{aligned} \tag{2}$$

It is assumed that a friction term exists, which is linearly related to the pitch angular velocity $\dot{\alpha}$ and acts in the opposite direction. The moments M_L and M_f , caused by the pitch motor lift F_L and friction respectively, are derived in Equation 3. F_L is defined in terms of the system state ω using Equation 2.

$$\begin{aligned} M_L &= F_L \cdot l_L \\ M_L &= k_1 \cdot l_L \cdot \omega^2 \\ M_f &= -k_2 \cdot \dot{\alpha} \end{aligned} \tag{3}$$

Finally, the sum of moments is derived, which is equal to the pitch angular acceleration $\ddot{\alpha}$ multiplied by the moment of inertia J_α . Combining all of this leads to the first differential equation, shown in Equation 4.

$$\begin{aligned} \sum M &= M_f + M_g + M_L = J_\alpha \cdot \ddot{\alpha} \\ J_\alpha \cdot \ddot{\alpha} &= -k_2 \cdot \dot{\alpha} - m \cdot g \cdot l_m \cdot \sin(\alpha) + k_1 \cdot l_L \cdot \omega^2 \\ \ddot{\alpha} &= -\frac{k_2}{J_\alpha} \cdot \dot{\alpha} - \frac{m \cdot g \cdot l_m}{J_\alpha} \cdot \sin(\alpha) + \frac{k_1 \cdot l_L}{J_\alpha} \cdot \omega^2 \end{aligned} \tag{4}$$

The same process is applied to the sum of moments of the motor propeller in Equation 5. Here, a linear relation is assumed between the voltage applied to the DC motor and the subsequently delivered torque. A drag term M_D caused by air resistance is also assumed. Combining these leads to the second differential equation, describing the change in motor angular velocity ω .

$$\begin{aligned} M_D &= -k_4 \cdot \omega^2 \\ M_u &= k_3 \cdot u \\ \sum M &= M_D + M_u = J_\omega \cdot \dot{\omega} \\ J_\omega \cdot \dot{\omega} &= -k_4 \cdot \omega^2 + k_3 \cdot u \\ \dot{\omega} &= -\frac{k_4}{J_\omega} \cdot \omega^2 + \frac{k_3}{J_\omega} \cdot u \end{aligned} \tag{5}$$

The final differential equation describing the change in pitch angle α is simply:

$$\ddot{\alpha} = \dot{\alpha}$$

The three nonlinear differential equations describing the 1-DoF helicopter system are displayed together in Equation 6. These are further simplified by combining the linear coefficients into a single coefficient for each term.

$$\begin{aligned}\ddot{\alpha} &= -\frac{k_2}{J_\alpha} \cdot \dot{\alpha} + -\frac{m \cdot g \cdot l_m}{J_\alpha} \cdot \sin(\alpha) + \frac{k_1 \cdot l_L}{J_\alpha} \cdot \omega^2 \\ \dot{\alpha} &= \dot{\alpha} \\ \dot{\omega} &= -\frac{k_4}{J_\omega} \cdot \omega^2 + \frac{k_3}{J_\omega} \cdot u\end{aligned}\tag{6}$$

$$\begin{aligned}\ddot{\alpha} &= -a_1 \cdot \dot{\alpha} + -a_2 \cdot \sin(\alpha) + a_3 \cdot \omega^2 \\ \dot{\alpha} &= \dot{\alpha} \\ \dot{\omega} &= -a_4 \cdot \omega^2 + b_1 \cdot u\end{aligned}$$

If initial guesses for these coefficients are needed, it is useful to describe them in their respective parameters. This is derived and shown in Equation 7, where k_1 is now an unknown friction coefficient and c_{motor} is the torque/voltage coefficient of the DC motor, which might be available in documentation.

$$\begin{aligned}a_1 &= \frac{k_1}{J_\alpha} & a_2 &= \frac{m \cdot g \cdot l_m}{J_\alpha} \\ a_3 &= \frac{l_L \cdot c_l \cdot \rho \cdot A \cdot r^2}{2 \cdot J_\alpha} & a_4 &= \frac{c_l \cdot \rho \cdot A \cdot r^2}{2 \cdot J_\omega} \\ b_1 &= \frac{c_{motor}}{J_\omega}\end{aligned}\tag{7}$$

2.3 Linear equations of motion

There are benefits to having a linear model. It will for example make the estimation procedure easier and the model will be more intuitive. For this reason the model is linearised around operating points. The states at these operating points can then be easily obtained and used for identification.

$$\begin{aligned}\tilde{x} &= (x - x_{op}) = \begin{bmatrix} \tilde{\alpha} \\ \tilde{\alpha} \\ \tilde{\omega} \end{bmatrix} = \begin{bmatrix} \dot{\alpha} - \dot{\alpha}_{op} \\ \alpha - \alpha_{op} \\ \omega - \omega_{op} \end{bmatrix} \\ \tilde{u} &= (u - u_{op})\end{aligned}$$

$$\begin{aligned}f_{\dot{\alpha}}(x) &\approx f_{\dot{\alpha}}(x_{op}) + \left. \frac{\partial f_{\dot{\alpha}}(x)}{\partial x} \right|_{x=x_{op}} (x - x_{op}) \\ \ddot{\alpha} &\approx 0 - a_1(\dot{\alpha} - \dot{\alpha}_{op}) - a_2 \cos(\alpha_{op})(\alpha - \alpha_{op}) + 2a_3 \omega_{op}(\omega - \omega_{op}) \\ \ddot{\alpha} &\approx -a_1 \dot{\tilde{\alpha}} - a_2 \cos(\alpha_{op}) \tilde{\alpha} + 2a_3 \omega_{op} \tilde{\omega} \\ \dot{\alpha} &= \dot{\alpha} \implies \dot{\tilde{\alpha}} = \dot{\tilde{\alpha}}\end{aligned}\tag{8}$$

$$\begin{aligned}
f_\omega(x) &\approx f_\omega(x_{op}) + \left. \frac{\partial f_\omega(x)}{\partial x} \right|_{x=x_{op}} (x - x_{op}) \\
\dot{\omega} &\approx 0 - 2a_4\omega_{op}(\omega - \omega_{op}) + b_1(u - u_{op}) \\
\dot{\omega} &\approx -2a_4\omega_{op}\tilde{\omega} + b_1\tilde{u}
\end{aligned} \tag{9}$$

These linear approximations of the equations of motion are used to form the linearised state space representation. After the operating points α_{op} and ω_{op} are inserted, the A matrix will solely have linear terms. The C matrix is also shown below. Only the second and third state can be measured. Since, the D matrix consists out of zeros it is omitted.

$$\begin{aligned}
\dot{\tilde{x}} &= \begin{bmatrix} -a_1\dot{\tilde{\alpha}} - a_2\cos(\alpha_{op})\tilde{\alpha} + 2a_3\omega_{op}\tilde{\omega} \\ \dot{\tilde{\alpha}} \\ -2a_4\omega_{op}\tilde{\omega} + b_1\tilde{u} \end{bmatrix} \\
&= \begin{bmatrix} -a_1 & -a_2\cos(\alpha_{op}) & 2a_3\omega_{op} \\ 1 & 0 & 0 \\ 0 & 0 & -2a_4\omega_{op} \end{bmatrix} \tilde{x} + \begin{bmatrix} 0 \\ 0 \\ b_1 \end{bmatrix} \tilde{u} \\
\tilde{y} = \begin{bmatrix} \tilde{\alpha} \\ \tilde{\omega} \end{bmatrix} &= \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tilde{x}
\end{aligned} \tag{10}$$

It is possible to simplify this further down to two states by incorporating the final state ω directly into the first state $\dot{\alpha}$. However, this is not done to keep the intuition intact and provide more insight on the relation between control input u and the motor angular velocity ω . The model shown in Equation 10 will be used throughout the rest of this project.

3 System identification

The derived in Chapter 2 state space shown in Equation 10 still holds five unknown coefficients. These coefficients have to be identified to fully define the system. In this chapter the identification process is discussed.

3.1 Identification strategy

The identification of the system using the readily available knowledge is a typical grey-box model estimation problem. The grey-box model estimation functions of **MATLAB** will be used. First, to fit the model to reality, data needs to be collected. This data is then processed to increase its effectiveness, for example the spikes in pitch angle are attenuated by **MATLAB**'s `movmean(.)` function. Then, all of the measured output is passed through a low-pass filter by the function `lowpass(.)` to clear the high frequency noise. For the filter parameters, Fast Fourier Transform of the data is plotted and the cutoff frequency is selected such that frequencies with lower magnitudes are removed. Moreover, to allow for a sharper decrease in the magnitude plot of the filter, the "steepness" parameter is set as 0.99 and the "stop band attenuation" is set as 80dB rather than the default of 60dB. The FFT result and the processed measurements of a chirp response of magnitude 1 is displayed on Figure 5 and Figure 6. It can be seen that the high frequency noise is mostly removed. After filtering, the data is then used together with the model knowledge to the grey-box estimation functions and derived model is validated.

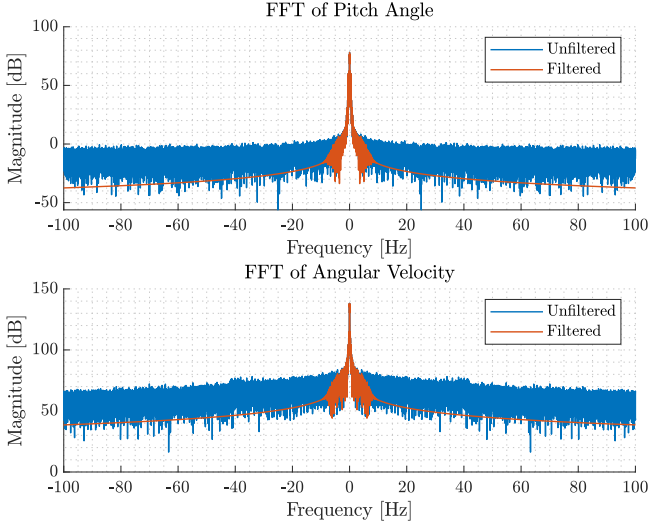


Figure 5: FFT of the Filtered and Unfiltered Measurements

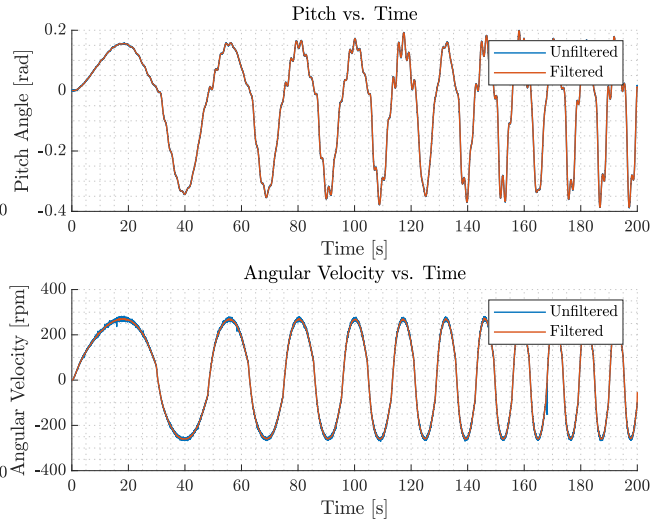


Figure 6: Filtered and Unfiltered Measurements

3.2 Excitation signals

The identification is done by collecting step and chirp responses. Step responses help extract time domain characteristics of the system while the chirp responses display how the system reacts to varying frequencies. After applying chirp responses around zero, it is observed that the output for positive and negative angles display different oscillatory patterns. Therefore, the identification for the two directions are done separately by collecting data in two different directions. From now on, the identified model with positive input will be called Positive Input Model and the negative direction will be called the Negative Input Model.

10 step responses for both directions with equally spaced input magnitudes ranging from 0.1 to 1 are recorded. Additionally, the system is applied 4 chirp signals with different amplitudes of 0.1, 0.2, 0.3 and 0.4, and frequency range [0.01, 0.1] around an offset of magnitude 0. The input and output data collected are separately stored in cell arrays for each direction and each cell array is processed through the `greyest(.)` function at once to produce a global model for that direction.

3.3 Grey-box Model Estimation

For the estimation itself, the linear model is defined as a function with matrices defined below:

$$\begin{aligned} A &= \begin{bmatrix} -a_1 & -a_2 & a_3 \\ 1 & 0 & 0 \\ 0 & 0 & -a_4 \end{bmatrix} & B &= \begin{bmatrix} 0 \\ 0 \\ b \end{bmatrix} \\ C &= \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} & D &= \begin{bmatrix} 0 \\ 0 \end{bmatrix} \end{aligned} \quad (11)$$

The linear model with estimable parameters is inserted in `idgrey(.)` together with the initial guesses. First, the initial guesses are set as 1. Then, the first estimation is rounded and the initial guesses are changed to these numbers for better convergence. This is an effective way to obtain proper initial guesses without the need to know all the physical parameters. `idgrey(.)` will output a system which can be used for estimation using `greyest(.)`.

Now, the coefficients can be estimated using the processed system responses to the excitation signals described in Section 3.2 and the identifiable system computed by `idgrey(.)`. The `greyest(.)` function will try to output a system that best fits the model, by minimising the error between estimated model response and the actual response data. This fully defined model is now ready to be validated.

3.4 Model Validation

Validation of the models are done through a separate dataset of 4 chirp responses of the system. The global state space models obtained are simulated via the state update block generated on **Simulink** and the outputs are compared to the real data via `goodnessToFit(.)` function with Normalized Root Mean Squared Error as the cost function (NRMSE). The output are compared for each of the 4 cases and the average goodness to fit is calculated via the following equation:

$$\text{Fit (\%)} = (1 - \text{NRMSE}) \cdot 100$$

3.5 Results

The identified models are presented in this section with comparisons to the simulation results as well as NRMSE values.

POSITIVE INPUT MODEL

The identified state space model matrices is presented below:

$$A = \begin{bmatrix} -0.1501 & -10.2307 & 0.0061 \\ 1 & 0 & 0 \\ 0 & 0 & -2.7850 \end{bmatrix} \quad B = \begin{bmatrix} 0 \\ 0 \\ 1125.5 \end{bmatrix}$$

$$C = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad D = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

The validation results are calculated as:

NRMSE of the Pitch Angle: 81.27%

NRMSE of the Motor Angular Velocity: 85.60%

A sample output with the input given in Figure 7 is displayed on Figure 8. It can be seen that while approaching the lower frequency oscillatory response, the simulation of the model is close to the actual data. However, in the oscillatory state it fails to capture small changes in angle. For the motor angular velocity, the simulation seems consistent with the real measurements. When the NRMSE values are considered, the model is acceptable and ready for the design of the state estimator.

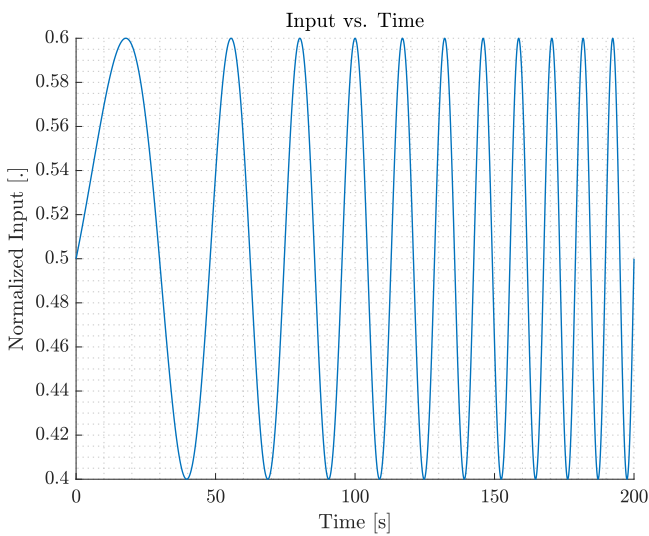


Figure 7: Sample Positive Input Applied

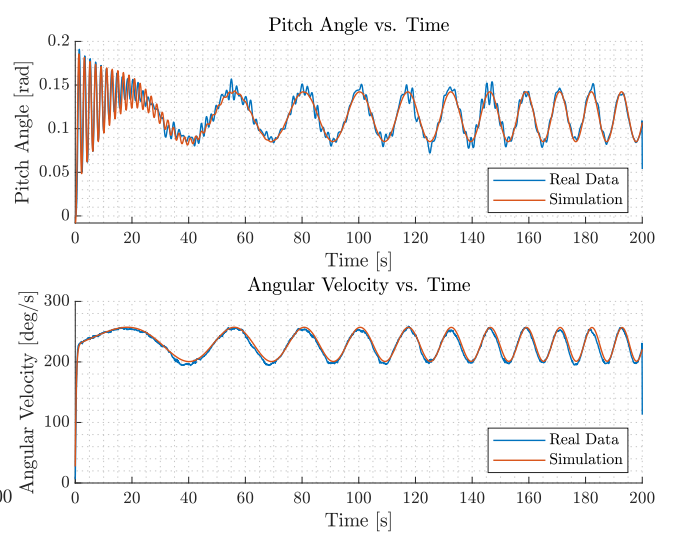


Figure 8: Real Data vs. Simulation for Positive Input

NEGATIVE INPUT MODEL

The identified state space model matrices is presented below:

$$A = \begin{bmatrix} -0.1991 & -9.2356 & 0.0124 \\ 1 & 0 & 0 \\ 0 & 0 & -2.9780 \end{bmatrix} \quad B = \begin{bmatrix} 0 \\ 0 \\ 1162.6 \end{bmatrix}$$

$$C = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad D = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

The validation results are calculated as:

NRMSE of the Pitch Angle: 80.46%

NRMSE of the Motor Angular Velocity: 87.13%

A sample output with the input given in Figure 9 is displayed on Figure 10. Similar to the positive input model, the pitch angle can follow the oscillation frequency in the first few seconds. It fails to capture the magnitude. As the constant magnitude oscillations continue, the model approaches the real data. Angular velocity simulation also matches the constant magnitude oscillatory response. These observations are validated numerically with the NRMSE values. Thus, the model can be used for state estimator design of the next section.

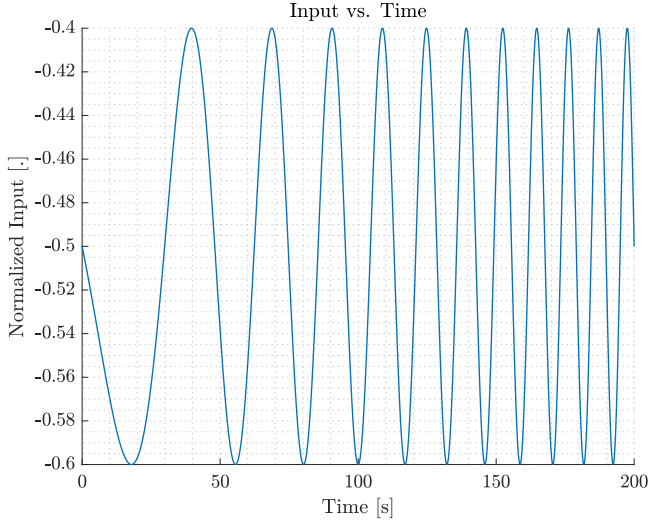


Figure 9: Sample Negative Input Applied

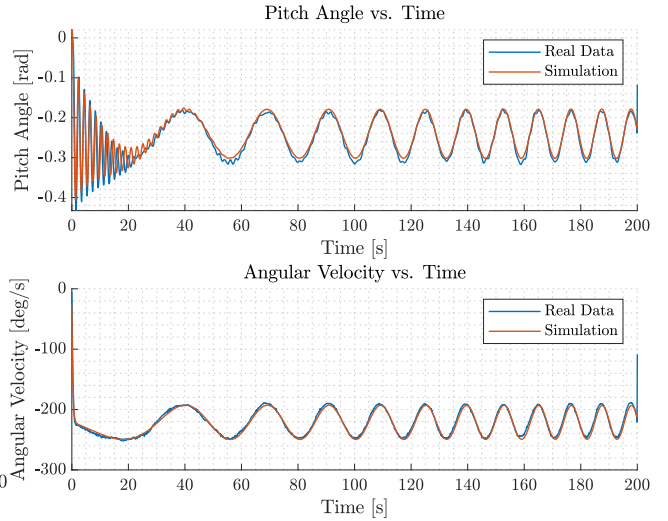


Figure 10: Real Data vs. Simulation for Negative Input

4 Model estimation

The helicopter system allows for two sensor measurements: pitch angle and motor angular velocity. However, the first state, pitch angle velocity cannot be observed via the hardware, thus, to implement a state feedback controller, that state should be estimated. The estimation is done by using the Kalman filter function of MATLAB and implementing the Kalman gain as the state observer gain. Kalman filters assume the noisy system to have zero mean noise with process and measurement covariance matrices, Q and R . Then it runs an optimisation to minimize the error between the actual system and the noisy system. Since the error can also be defined as a zero mean noise, the optimization can be run to find a gain to reduce the error between the real output and the noisy output. When simplified, the Kalman filter gain presents the following state update equation with the actual output y_k , estimated state $\hat{x}_k$ and Kalman gain K_k :

$$\hat{x}_k = A\hat{x}_{k-1} + Bu_k + K_k (y_k - C(A\hat{x}_{k-1} + Bu_k))$$

This equation is realized on Simulink via simple gain and sum blocks by calculating the Kalman gain offline from the identified state matrices.

By tuning Q and R matrices, a Kalman gain that allows the estimation to follow real data can be designed. Q matrix is taken as having only diagonal nonzero elements while R is a scalar. While tuning, the second and

third states with measurements already available are given much lower weights than the first state since there is no measurement available and it is more important for the first state error to be minimized. In order to compare the outputs, the derivative of the measurements of the second state are taken and passed through a low-pass filter to gather information about the real values of the first state.

ESTIMATOR FOR THE POSITIVE INPUT MODEL

The input and comparison of the states are presented in Figure 11, Figure 12, Figure 13 and Figure 14. Even though the first two states can be followed, the last state estimation has slight error for an oscillatory input which can be lowered by tuning the filter further.

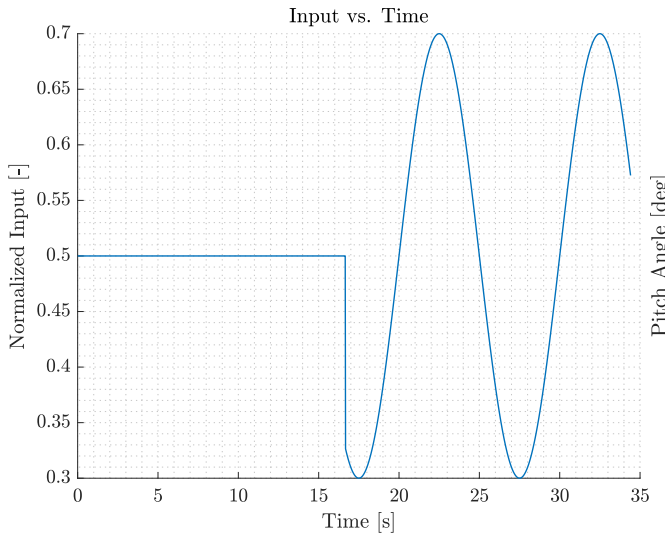


Figure 11: Positive Input vs. Time

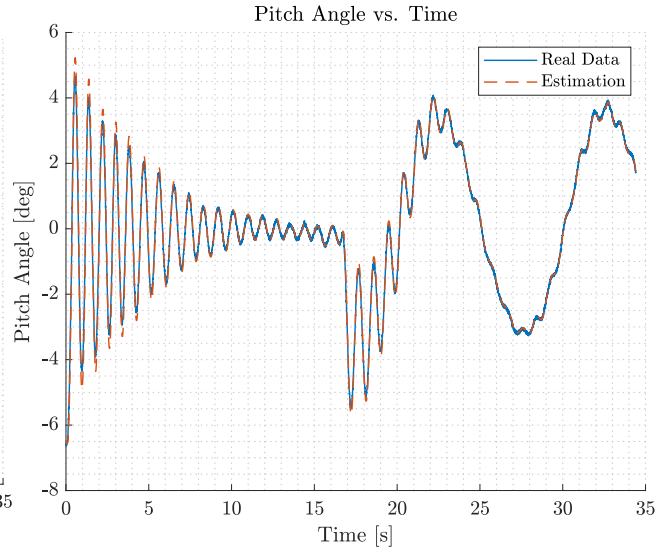


Figure 12: Pitch Angle vs. Time for Positive Input

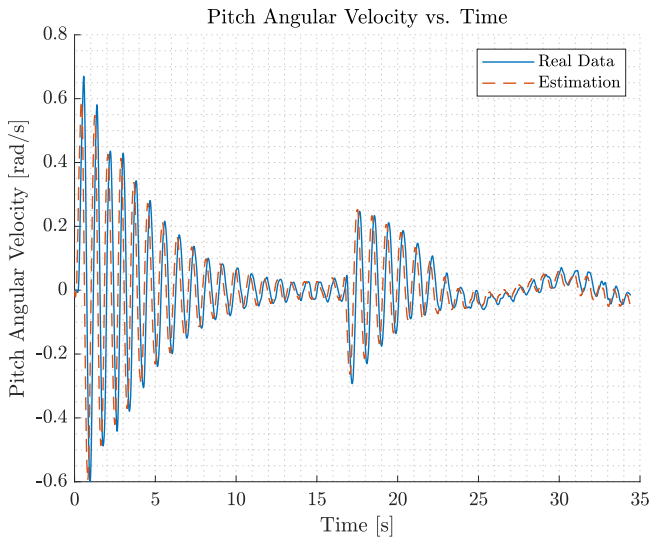


Figure 13: Pitch Angular Velocity vs. Time for Positive Input

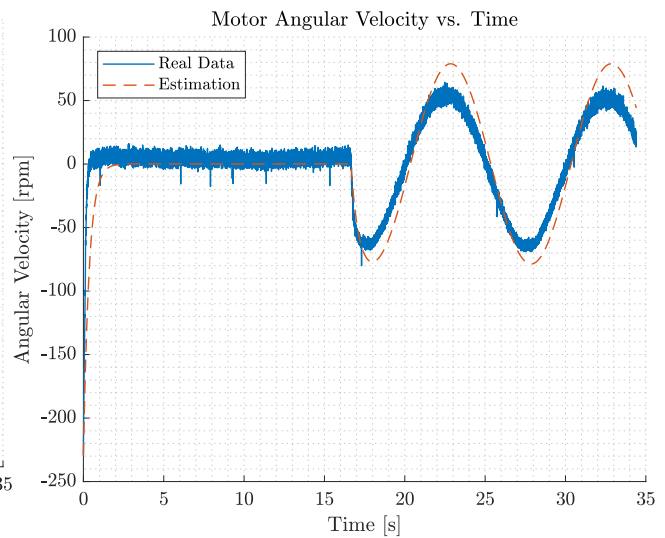


Figure 14: Pitch Angle vs. Time for Positive Input

ESTIMATOR FOR THE NEGATIVE INPUT MODEL

The input and comparison of the states are shown in Figure 11, Figure 12, Figure 13 and Figure 14. Similar to the positive model an error is present in the last state estimation has slight error for an oscillatory input. Further tuning is required to lower the error.

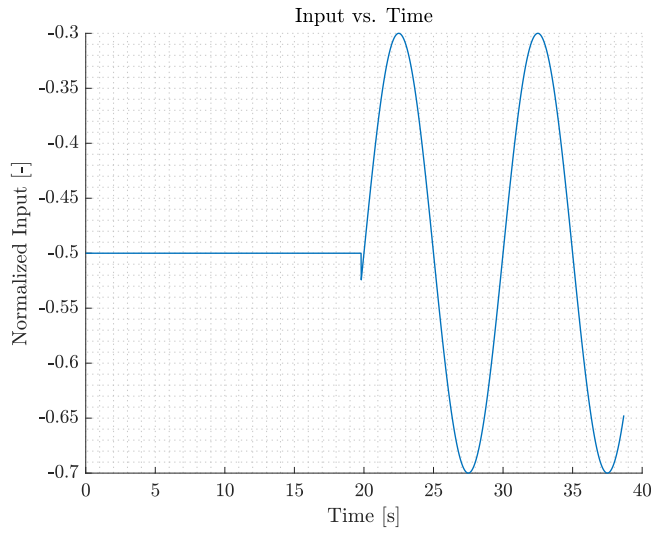


Figure 15: Negative Input vs. Time

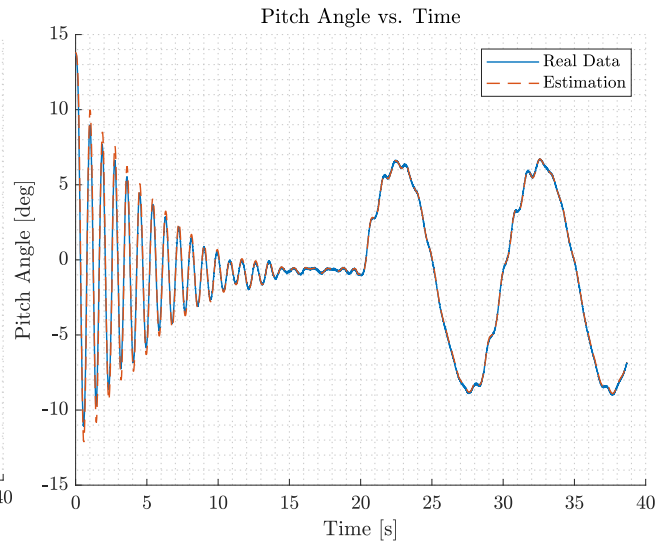


Figure 16: Pitch Angle vs. Time for Negative Input

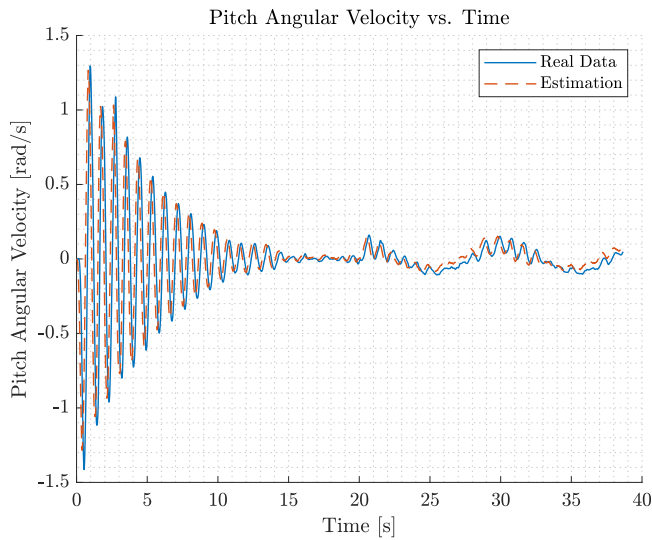


Figure 17: Pitch Angular Velocity vs. Time for Negative Input

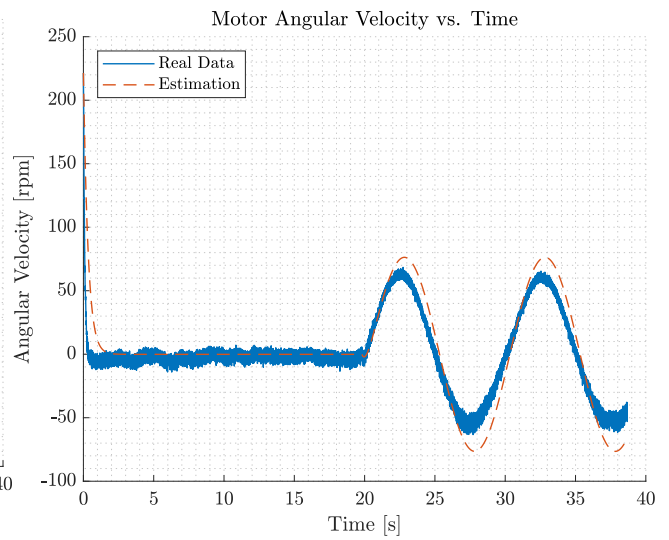


Figure 18: Pitch Angle vs. Time for Negative Input

5 Control design

Now a fully defined model is obtained, controllers can be designed to reach the control objectives. In this chapter the control objective and the choice of control architecture is explained. The design of the individual controllers is discussed, and finally the performance of the controllers is evaluated.

5.1 Objectives

As mentioned in earlier in Chapter 1.1, the objective is to control the pitch angle of the 1-DoF helicopter. The first goal is to have the pitch of the helicopter stay stationary at a specified angle. It should then reject any introduced disturbance, i.e. the helicopter should return to the specified pitch angle after being pushed to a different angle by an outside force. The faster the specified angle is regained, the better the performance.

If the disturbance rejection is managed, an attempt can be made to do reference tracking of the pitch angle. Here, the pitch angle has to follow a specified path, i.e. the nose of the helicopter moving up and down according to an arbitrarily predefined motion. This path can for example be a sinusoidal signal. The fit of the actual path to the reference path indicates the performance. Next to this, a disturbance (a small push) can be applied. The time it takes to regain tracking shows performance.

5.2 Choice of architecture

For this system two state-feedback controller architectures are investigated. The first is Linear-Quadratic Regression (LQR) control including an additional integrator, creating Linear-Quadratic-Integral (LQI) control. For the second controller, a Model Predictive Control (MPC) approach will be used.

LQR control obtains the optimal control action by minimising the objective function shown below in Equation 5.2 offline.

$$J(u) = \int_0^{\infty} (x^T Q x + u^T R u + 2x^T N u) dt \quad (12)$$

Here the Q matrix hold the weights that penalize errors of the system states. Increasing a weight corresponds to faster convergence of the respective state. The R matrix puts a penalty on control effort, ensuring that the control inputs are not excessively large, which, next to being safer, can be beneficial for energy efficiency and actuator longevity. In this case N is assumed to be zero. There are a number of advantages of using LQR, for example:

- Optimal Performance: LQR finds the optimal control action by minimising a cost function that balances state errors and control effort.
- Stability: LQR design guarantees stability of LTI system (if controllable and observable).
- Easy implementation: The complexity of LQR control is manageable. This lowers the probability of coding errors and saves time. It also makes tuning the controller by adjusting the weight matrices more intuitive.
- Speed: Since the LQR gains are computed offline, there is no need to perform extensive calculation for each time step. This means less computing power is necessary and the time step can be decreased, enabling a faster response to sudden changes (disturbances).

A big disadvantage of LQR is its handling of constraints. It does not take the system limits into account, which means additional techniques have to be applied to compensate, for example a saturation function. The control action might become suboptimal after these additional techniques are applied.

The integrator is added to remove the steady state error. It will also cause the controller to have a less aggressive response.

Alternatively, Model Predictive Control calculates the optimal control action at each time step. It does this by solving the objective function shown in Equation 5.2. The difference is it minimizes the state error and control effort for a specified number of time steps, the prediction horizon N . It predicts where the state will be for each time step within the prediction horizon and takes all of this into account when optimising. The final term, the terminal cost, puts weight on the state beyond the event horizon, ensuring the state error is minimal for future time steps as well.

$$J(x(t)) = \sum_{k=0}^{N-1} (x_k^T Q x_k + u_k^T R u_k) + V^{j-1}(x_N) \quad (13)$$

Some advantages of Model Predictive Control are:

- **Optimal Performance:** Similarly to LQR, MPC provides optimal control action while balancing state errors and control effort. However, the optimisation is now redone on each time step, adjusting control based on updated predictions.
- **Robustness:** By continuously updating, MPC adapts to newly introduced disturbances and model inaccuracies. This enhances reliability under changing conditions.
- **Constraints:** MPC ensures the system stays within predefined limits, by taking constraints into account when solving the optimisation problem.

The disadvantage of MPC lies in its complexity and required computing time. The high complexity might make it more difficult to set up, implement and debug. MPC works very well for slow systems, but as the time step gets smaller it may fail to find a optimal solution in time. This is especially challenging for older systems with limited computational resources. Also, a very accurate system model is required. If the model is inaccurate, the prediction errors will get propagate, getting worse with each time step further in the future.

5.3 Linear-Quadratic-Integral (LQI) control

The LQI gains are computed using MATLAB `dLQR(.)` and implemented in `simulink`, the corresponding files show exactly how.

The following plots show the response of the implemented LQI controller to an introduced disturbance (push). This is done for positive and negative pitch angles separately, because the models changes significantly for these states.

POSITIVE INPUT MODEL

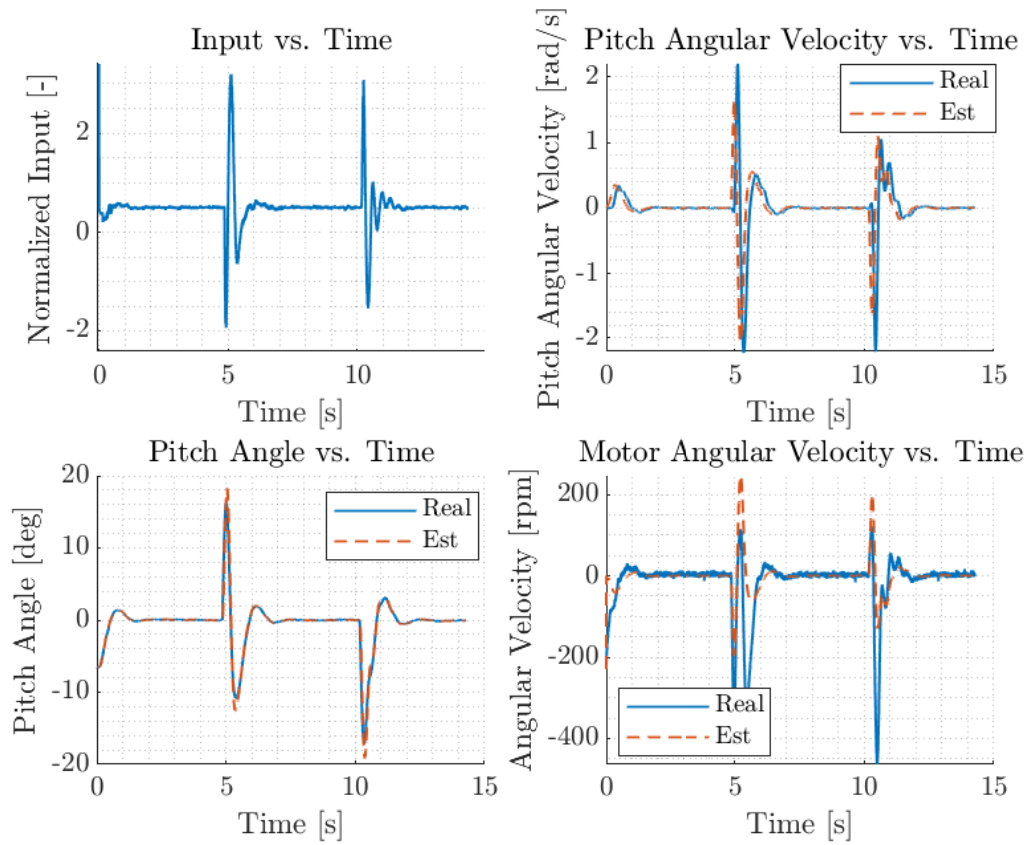


Figure 19: LQI Control Input and States for Positive Angles

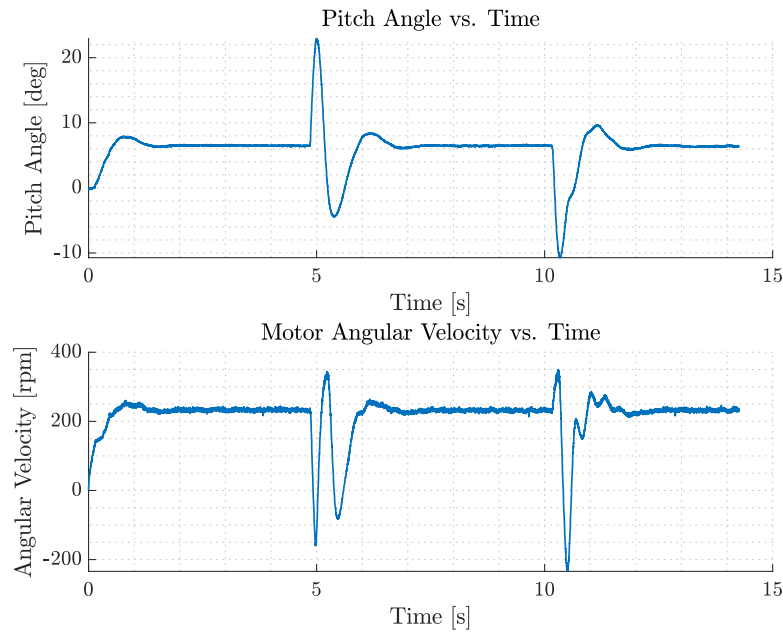


Figure 20: LQI Control Output for Positive Angles

NEGATIVE INPUT MODEL

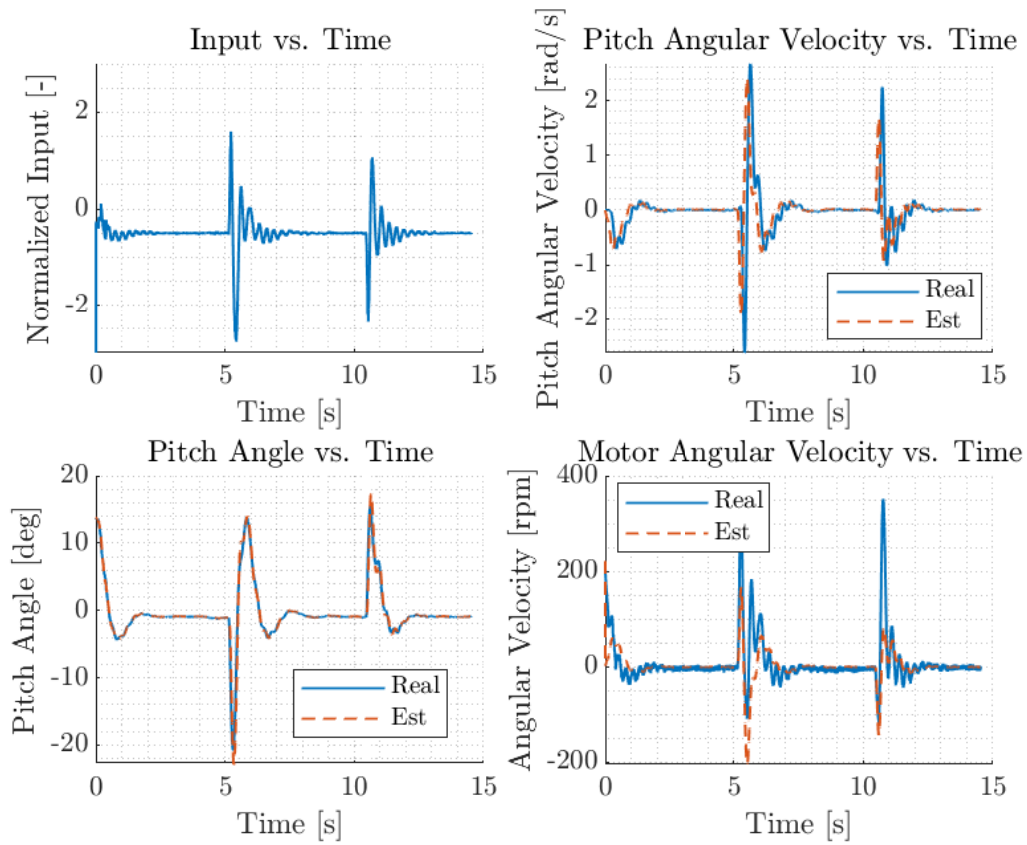


Figure 21: LQI Control Input and States for Negative Angles

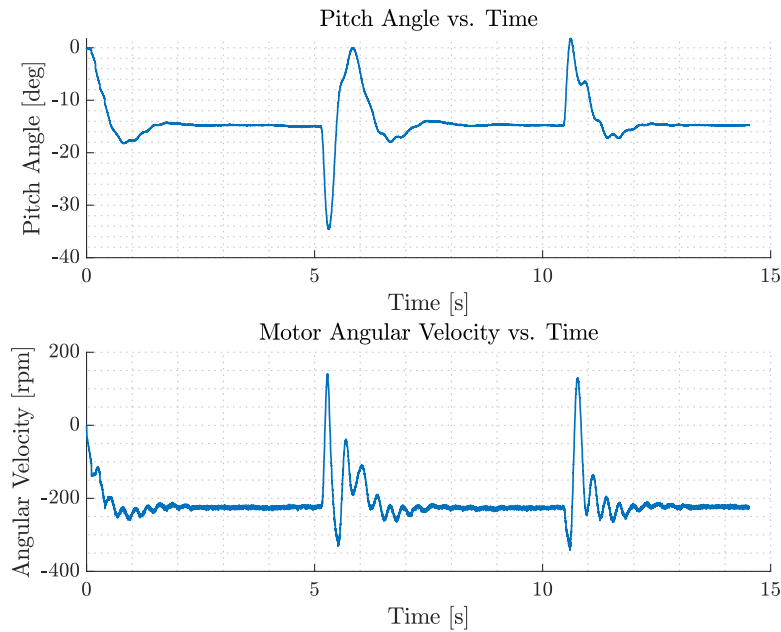


Figure 22: LQI Control Output for Negative Angles

The plots show that the disturbance is rejected well. The time it takes to regain the desired angle can possibly be improved by further fine-tuning the weight matrices.

5.4 Model Predictive Control (MPC)

For the Model Predictive Control approach the stabilizing solution of the Riccati equation was used as the terminal cost, ensuring stability. Two solvers were used, quadprog, which uses interior point, and MPCActiveSetSolver which uses active set. The main benefit of the latter is that "Linv", the lower-triangular Cholesky decomposition of the Hessian, can be computed offline, making the optimisation faster. Overall both solvers are reasonably quick and should show similar performance for small to medium sized optimisation problems. The MATLAB code and simulink model should provide more inside on the implementation.

POSITIVE INPUT MODEL

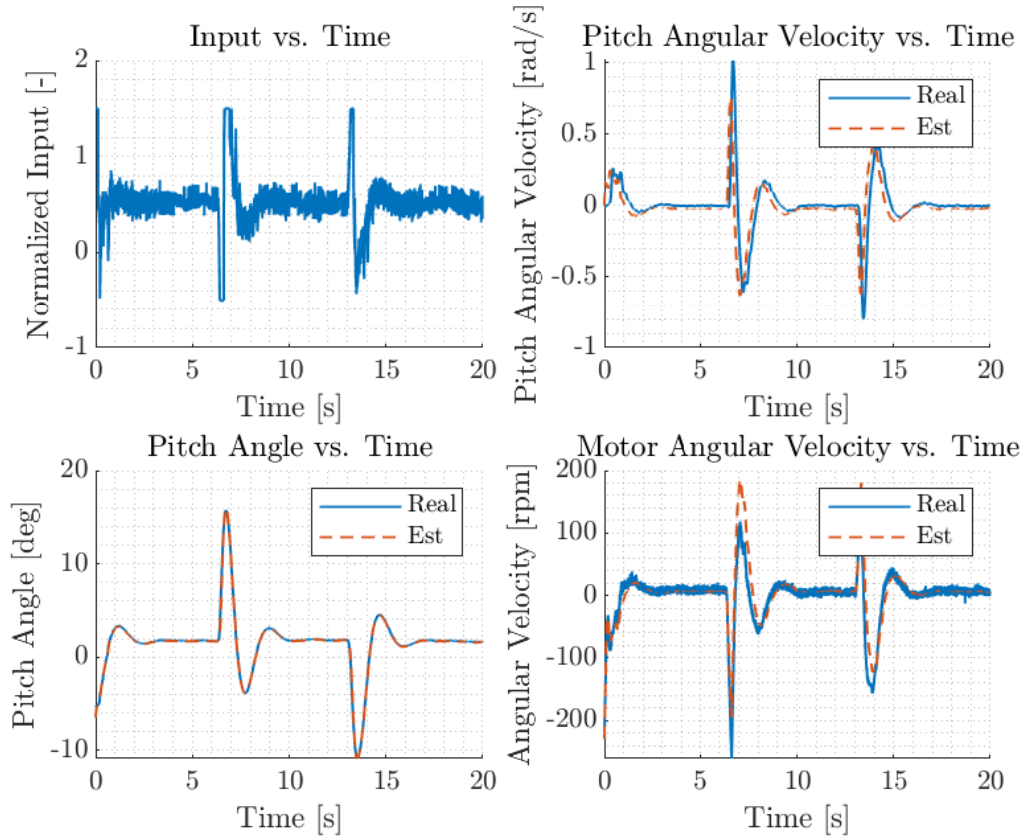


Figure 23: MPC Input and States for Positive Angles

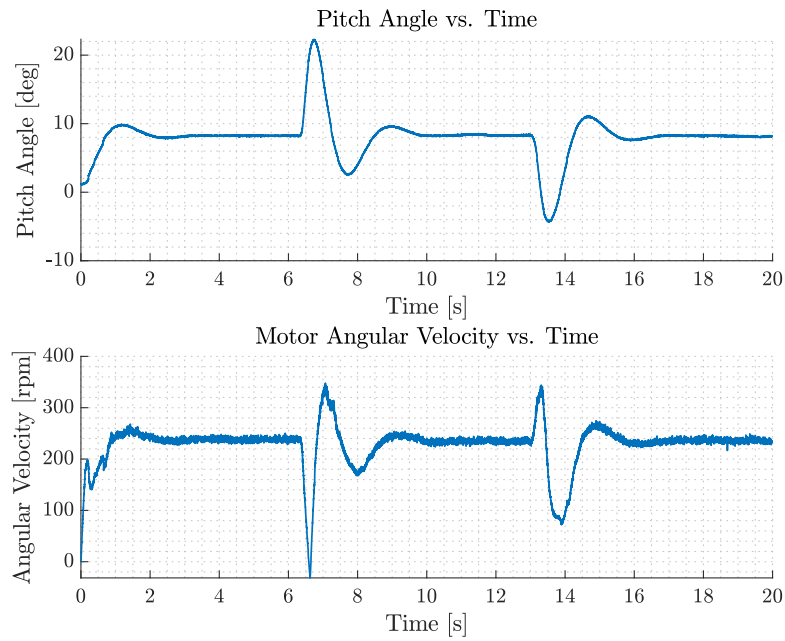


Figure 24: MPC Output for Positive Angles

NEGATIVE INPUT MODEL

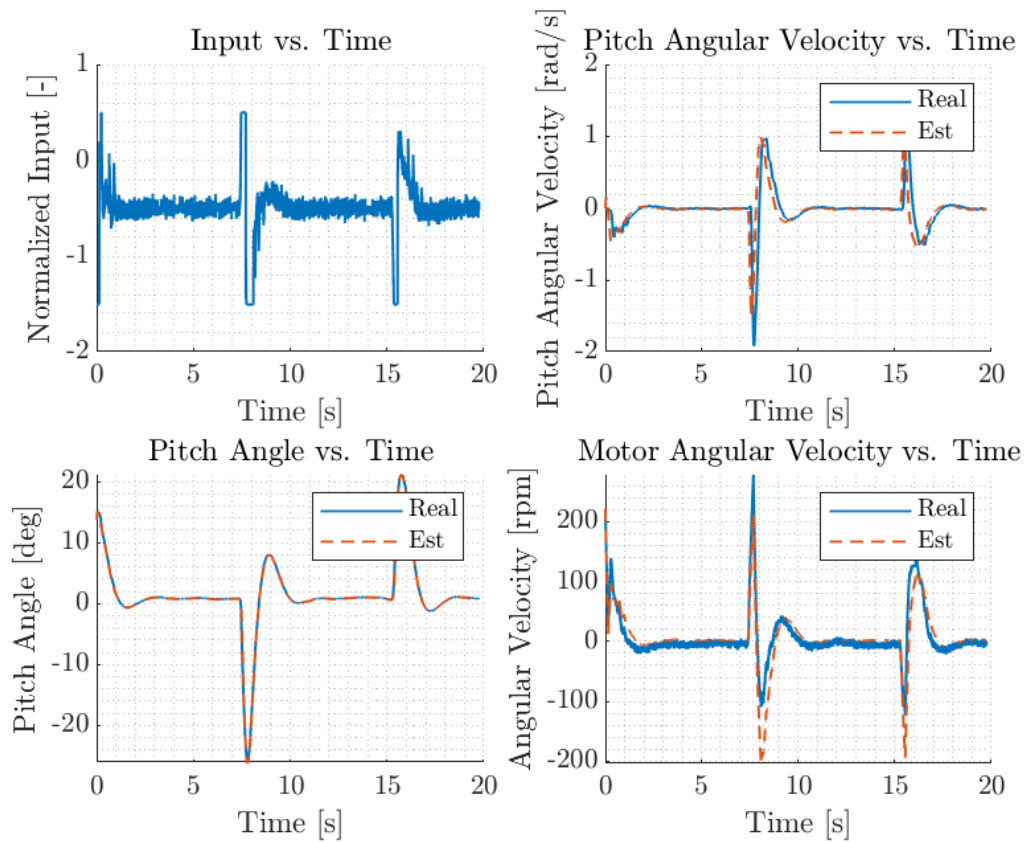


Figure 25: MPC Input and States for Negative Angles

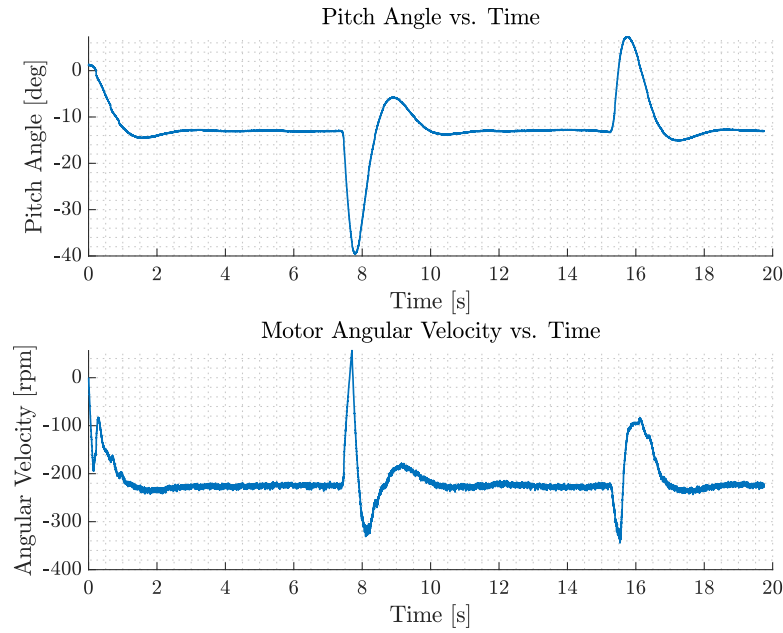


Figure 26: MPC Output for Negative Angles

The controller manages to reject the disturbance. This results already shows a significant improvement compared to the uncontrolled system. These results were obtained after very limited tuning, thus the maximum achievable performance is expected to show even better results.

Discussion

The most difficult part about this project has been the delayed start due to the organisation of the Control Robotics Fair. It is extremely difficult to make up for 2 to 3 weeks of time in an already tough period. Nonetheless, the obtained results are something to be proud of. Having functioning controllers was prioritised, and this was managed quite well. Unfortunately, there was not enough time to report all the design choices and findings. This is the main area of improvement. In short, the controlled systems are working well, but there is still an opportunity to fine-tune them into performing even better.

References

- [1] H. T. V. (HTV). (2024) Three axes of flight. [Online]. Available: <https://www.helicoptertrainingvideos.com/helicopter-training-videos/helicopter-aerodynamics/three-axes-of-flight/>